Chapter 2

Basics of Algorithm Analysis

Analyzing algorithms involves thinking about how their resource requirements—the amount of time and space they use—will scale with increasing input size. We begin this chapter by talking about how to put this notion on a concrete footing, as making it concrete opens the door to a rich understanding of computational tractability. Having done this, we develop the mathematical machinery needed to talk about the way in which different functions scale with increasing input size, making precise what it means for one function to grow faster than another.

We then develop running-time bounds for some basic algorithms, beginning with an implementation of the Gale-Shapley algorithm from Chapter 1 and continuing to a survey of many different running times and certain characteristic types of algorithms that achieve these running times. In some cases, obtaining a good running-time bound relies on the use of more sophisticated data structures, and we conclude this chapter with a very useful example of such a data structure: priority queues and their implementation using heaps.

2.1 Computational Tractability

A major focus of this book is to find efficient algorithms for computational problems. At this level of generality, our topic seems to encompass the whole of computer science; so what is specific to our approach here?

First, we will try to identify broad themes and design principles in the development of algorithms. We will look for paradigmatic problems and approaches that illustrate, with a minimum of irrelevant detail, the basic approaches to designing efficient algorithms. At the same time, it would be pointless to pursue these design principles in a vacuum, so the problems and

approaches we consider are drawn from fundamental issues that arise throughout computer science, and a general study of algorithms turns out to serve as a nice survey of computational ideas that arise in many areas.

Another property shared by many of the problems we study is their fundamentally *discrete* nature. That is, like the Stable Matching Problem, they will involve an implicit search over a large set of combinatorial possibilities; and the goal will be to efficiently find a solution that satisfies certain clearly delineated conditions.

As we seek to understand the general notion of computational efficiency, we will focus primarily on efficiency in running time: we want algorithms that run quickly. But it is important that algorithms be efficient in their use of other resources as well. In particular, the amount of *space* (or memory) used by an algorithm is an issue that will also arise at a number of points in the book, and we will see techniques for reducing the amount of space needed to perform a computation.

Some Initial Attempts at Defining Efficiency

The first major question we need to answer is the following: How should we turn the fuzzy notion of an "efficient" algorithm into something more concrete?

A first attempt at a working definition of *efficiency* is the following.

> Proposed Definition of Efficiency (1): *An algorithm is efficient if, when implemented, it runs quickly on real input instances.*

Let's spend a little time considering this definition. At a certain level, it's hard to argue with: one of the goals at the bedrock of our study of algorithms is solving real problems quickly. And indeed, there is a significant area of research devoted to the careful implementation and profiling of different algorithms for discrete computational problems.

But there are some crucial things missing from this definition, even if our main goal is to solve real problem instances quickly on real computers. The first is the omission of *where*, and *how well*, we implement an algorithm. Even bad algorithms can run quickly when applied to small test cases on extremely fast processors; even good algorithms can run slowly when they are coded sloppily. Also, what is a "real" input instance? We don't know the full range of input instances that will be encountered in practice, and some input instances can be much harder than others. Finally, this proposed definition above does not consider how well, or badly, an algorithm may *scale* as problem sizes grow to unexpected levels. A common situation is that two very different algorithms will perform comparably on inputs of size 100; multiply the input size tenfold, and one will still run quickly while the other consumes a huge amount of time.

So what we could ask for is a concrete definition of efficiency that is platform-independent, instance-independent, and of predictive value with respect to increasing input sizes. Before focusing on any specific consequences of this claim, we can at least explore its implicit, high-level suggestion: that we need to take a more mathematical view of the situation.

We can use the Stable Matching Problem as an example to guide us. The input has a natural "size" parameter N; we could take this to be the total size of the representation of all preference lists, since this is what any algorithm for the problem will receive as input. N is closely related to the other natural parameter in this problem: n, the number of men and the number of women. Since there are $2n$ preference lists, each of length n, we can view $N = 2n^2$, suppressing more fine-grained details of how the data is represented. In considering the problem, we will seek to describe an algorithm at a high level, and then analyze its running time mathematically as a function of this input size N.

Worst-Case Running Times and Brute-Force Search

To begin with, we will focus on analyzing the *worst-case* running time: we will look for a bound on the largest possible running time the algorithm could have over all inputs of a given size N, and see how this scales with N. The focus on worst-case performance initially seems quite draconian: what if an algorithm performs well on most instances and just has a few pathological inputs on which it is very slow? This certainly is an issue in some cases, but in general the worst-case analysis of an algorithm has been found to do a reasonable job of capturing its efficiency in practice. Moreover, once we have decided to go the route of mathematical analysis, it is hard to find an effective alternative to worst-case analysis. Average-case analysis—the obvious appealing alternative, in which one studies the performance of an algorithm averaged over "random" instances—can sometimes provide considerable insight, but very often it can also become a quagmire. As we observed earlier, it's very hard to express the full range of input instances that arise in practice, and so attempts to study an algorithm's performance on "random" input instances can quickly devolve into debates over how a random input should be generated: the same algorithm can perform very well on one class of random inputs and very poorly on another. After all, real inputs to an algorithm are generally not being produced from a random distribution, and so average-case analysis risks telling us more about the means by which the random inputs were generated than about the algorithm itself.

So in general we will think about the worst-case analysis of an algorithm's running time. But what is a reasonable analytical benchmark that can tell us whether a running-time bound is impressive or weak? A first simple guide

is by comparison with brute-force search over the search space of possible solutions.

Let's return to the example of the Stable Matching Problem. Even when the size of a Stable Matching input instance is relatively small, the *search space* it defines is enormous (there are $n!$ possible perfect matchings between n men and n women), and we need to find a matching that is stable. The natural "brute-force" algorithm for this problem would plow through all perfect matchings by enumeration, checking each to see if it is stable. The surprising punchline, in a sense, to our solution of the Stable Matching Problem is that we needed to spend time proportional only to N in finding a stable matching from among this stupendously large space of possibilities. This was a conclusion we reached at an *analytical level*. We did not implement the algorithm and try it out on sample preference lists; we reasoned about it mathematically. Yet, at the same time, our analysis indicated how the algorithm could be implemented in practice and gave fairly conclusive evidence that it would be a big improvement over exhaustive enumeration.

This will be a common theme in most of the problems we study: a compact representation, implicitly specifying a giant search space. For most of these problems, there will be an obvious brute-force solution: try all possibilities and see if any one of them works. Not only is this approach almost always too slow to be useful, it is an intellectual cop-out; it provides us with absolutely no insight into the structure of the problem we are studying. And so if there is a common thread in the algorithms we emphasize in this book, it would be the following alternative definition of efficiency.

> Proposed Definition of Efficiency (2): *An algorithm is efficient if it achieves qualitatively better worst-case performance, at an analytical level, than brute-force search.*

This will turn out to be a very useful working definition for us. Algorithms that improve substantially on brute-force search nearly always contain a valuable heuristic idea that makes them work; and they tell us something about the intrinsic structure, and computational tractability, of the underlying problem itself.

But if there is a problem with our second working definition, it is vagueness. What do we mean by "qualitatively better performance?" This suggests that we consider the actual running time of algorithms more carefully, and try to quantify what a reasonable running time would be.

Polynomial Time as a Definition of Efficiency

When people first began analyzing discrete algorithms mathematically—a thread of research that began gathering momentum through the 1960s—

a consensus began to emerge on how to quantify the notion of a "reasonable" running time. Search spaces for natural combinatorial problems tend to grow exponentially in the size N of the input; if the input size increases by one, the number of possibilities increases multiplicatively. We'd like a good algorithm for such a problem to have a better scaling property: when the input size increases by a constant factor—say, a factor of 2—the algorithm should only slow down by some constant factor C.

Arithmetically, we can formulate this scaling behavior as follows. Suppose an algorithm has the following property: There are absolute constants $c > 0$ and $d > 0$ so that on every input instance of size N, its running time is bounded by cN^d primitive computational steps. (In other words, its running time is at most proportional to N^d.) For now, we will remain deliberately vague on what we mean by the notion of a "primitive computational step"—but it can be easily formalized in a model where each step corresponds to a single assembly-language instruction on a standard processor, or one line of a standard programming language such as C or Java. In any case, if this running-time bound holds, for some c and d, then we say that the algorithm has a *polynomial running time*, or that it is a *polynomial-time algorithm*. Note that any polynomial-time bound has the scaling property we're looking for. If the input size increases from N to $2N$, the bound on the running time increases from cN^d to $c(2N)^d = c \cdot 2^d N^d$, which is a slow-down by a factor of 2^d. Since d is a constant, so is 2^d; of course, as one might expect, lower-degree polynomials exhibit better scaling behavior than higher-degree polynomials.

From this notion, and the intuition expressed above, emerges our third attempt at a working definition of efficiency.

Proposed Definition of Efficiency (3): *An algorithm is efficient if it has a polynomial running time.*

Where our previous definition seemed overly vague, this one seems much too prescriptive. Wouldn't an algorithm with running time proportional to n^{100}—and hence polynomial—be hopelessly inefficient? Wouldn't we be relatively pleased with a nonpolynomial running time of $n^{1+.02(\log n)}$? The answers are, of course, "yes" and "yes." And indeed, however much one may try to abstractly motivate the definition of efficiency in terms of polynomial time, a primary justification for it is this: *It really works.* Problems for which polynomial-time algorithms exist almost invariably turn out to have algorithms with running times proportional to very moderately growing polynomials like n, $n \log n$, n^2, or n^3. Conversely, problems for which no polynomial-time algorithm is known tend to be very difficult in practice. There are certainly exceptions to this principle in both directions: there are cases, for example, in

Table 2.1 The running times (rounded up) of different algorithms on inputs of increasing size, for a processor performing a million high-level instructions per second. In cases where the running time exceeds 10^{25} years, we simply record the algorithm as taking a very long time.

	n	$n \log_2 n$	n^2	n^3	1.5^n	2^n	$n!$
$n = 10$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	4 sec
$n = 30$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	< 1 sec	18 min	10^{25} years
$n = 50$	< 1 sec	< 1 sec	< 1 sec	< 1 sec	11 min	36 years	very long
$n = 100$	< 1 sec	< 1 sec	< 1 sec	1 sec	12,892 years	10^{17} years	very long
$n = 1{,}000$	< 1 sec	< 1 sec	1 sec	18 min	very long	very long	very long
$n = 10{,}000$	< 1 sec	< 1 sec	2 min	12 days	very long	very long	very long
$n = 100{,}000$	< 1 sec	2 sec	3 hours	32 years	very long	very long	very long
$n = 1{,}000{,}000$	1 sec	20 sec	12 days	31,710 years	very long	very long	very long

which an algorithm with exponential worst-case behavior generally runs well on the kinds of instances that arise in practice; and there are also cases where the best polynomial-time algorithm for a problem is completely impractical due to large constants or a high exponent on the polynomial bound. All this serves to reinforce the point that our emphasis on worst-case, polynomial-time bounds is only an abstraction of practical situations. But overwhelmingly, the concrete mathematical definition of polynomial time has turned out to correspond surprisingly well in practice to what we observe about the efficiency of algorithms, and the tractability of problems, in real life.

One further reason why the mathematical formalism and the empirical evidence seem to line up well in the case of polynomial-time solvability is that the gulf between the growth rates of polynomial and exponential functions is enormous. Suppose, for example, that we have a processor that executes a million high-level instructions per second, and we have algorithms with running-time bounds of n, $n \log_2 n$, n^2, n^3, 1.5^n, 2^n, and $n!$. In Table 2.1, we show the running times of these algorithms (in seconds, minutes, days, or years) for inputs of size $n = 10, 30, 50, 100, 1{,}000, 10{,}000, 100{,}000,$ and 1,000,000.

There is a final, fundamental benefit to making our definition of efficiency so specific: it becomes negatable. It becomes possible to express the notion that *there is no efficient algorithm for a particular problem*. In a sense, being able to do this is a prerequisite for turning our study of algorithms into good science, for it allows us to ask about the existence or nonexistence of efficient algorithms as a well-defined question. In contrast, both of our

previous definitions were completely subjective, and hence limited the extent to which we could discuss certain issues in concrete terms.

In particular, the first of our definitions, which was tied to the specific implementation of an algorithm, turned efficiency into a moving target: as processor speeds increase, more and more algorithms fall under this notion of efficiency. Our definition in terms of polynomial time is much more an absolute notion; it is closely connected with the idea that each problem has an intrinsic level of computational tractability: some admit efficient solutions, and others do not.

2.2 Asymptotic Order of Growth

Our discussion of computational tractability has turned out to be intrinsically based on our ability to express the notion that an algorithm's worst-case running time on inputs of size n grows at a rate that is at most proportional to some function $f(n)$. The function $f(n)$ then becomes a bound on the running time of the algorithm. We now discuss a framework for talking about this concept.

We will mainly express algorithms in the pseudo-code style that we used for the Gale-Shapley algorithm. At times we will need to become more formal, but this style of specifying algorithms will be completely adequate for most purposes. When we provide a bound on the running time of an algorithm, we will generally be counting the number of such pseudo-code steps that are executed; in this context, one *step* will consist of assigning a value to a variable, looking up an entry in an array, following a pointer, or performing an arithmetic operation on a fixed-size integer.

When we seek to say something about the running time of an algorithm on inputs of size n, one thing we could aim for would be a very concrete statement such as, "On any input of size n, the algorithm runs for at most $1.62n^2 + 3.5n + 8$ steps." This may be an interesting statement in some contexts, but as a general goal there are several things wrong with it. First, getting such a precise bound may be an exhausting activity, and more detail than we wanted anyway. Second, because our ultimate goal is to identify broad classes of algorithms that have similar behavior, we'd actually like to classify running times at a coarser level of granularity so that similarities among different algorithms, and among different problems, show up more clearly. And finally, extremely detailed statements about the number of steps an algorithm executes are often—in a strong sense—meaningless. As just discussed, we will generally be counting steps in a pseudo-code specification of an algorithm that resembles a high-level programming language. Each one of these steps will typically unfold into some fixed number of primitive steps when the program is compiled into